

class121 树的直径 笔记

树上问题专题四。前置：链式前向星建图（059）、树形DP（078/079）、递归改迭代（118）、并查集与集合打标签（056/057）、单调队列（054）。

树的直径：定义、两种求法、两个结论

定义：树上距离最远的两个点形成的路径长度。

两种求法：

	两次 DFS	树形 DP
适用范围	边权非负	所有树（含负边）
遍历次数	两遍	一遍
能得到路径/端点信息	能（last 数组）	只能得到长度

两个结论（边权都为正时成立）：

1. 树上有多条直径时，它们一定**共用中间的公共部分**（一个公共点或一段公共路径）；
2. 树上任意一点，离它最远的点里，**直径的两个端点至少有一个在其中**。

题目一：树的直径模版（U81904，边权可能为负）

题面

给定一棵树，边权可能为负，求直径长度。

写法 1：两遍 DFS

核心思路

第一次 DFS 从任意点（1 号）出发，找离它最远的点 `start`；第二次 DFS 从 `start` 出发，找离它最远的点 `end`，直径 = `dist[end]`。DFS 过程中顺手维护 `last[i]`（从当前头出发，i 的上一个节点），就能反推出整条直径路径。

坑点

- **只适用于边权非负**。有负边时两次 DFS 会求错（第一次找到的最远点不是直径端点），本题会有用例过不了。
- 好处是能得到路径上所有点，后面的题（公共部分、巡逻、消防）都要用到。

```
1  const int MAXN = 500001;
2  int n, cnt, start, fin, diameter;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1], weight[MAXN << 1];
4  int dist[MAXN], last[MAXN];
5
6  void dfs(int u, int f, int w) {
7      last[u] = f;
8      dist[u] = dist[f] + w;
9      for (int e = head[u]; e != 0; e = nxt[e])
10         if (to[e] != f) dfs(to[e], u, weight[e]);
11 }
```

```

11 }
12
13 void road() {
14     dfs(1, 0, 0);
15     start = 1;
16     for (int i = 2; i <= n; i++) if (dist[i] > dist[start]) start = i;
17     dfs(start, 0, 0);
18     fin = 1;
19     for (int i = 2; i <= n; i++) if (dist[i] > dist[fin]) fin = i;
20     diameter = dist[fin];
21 }

```

写法 2：树形 DP

核心思路

每个节点维护两个信息：

- `dist[u]`：从 `u` **必须往下走**（可不选任何边）能走出的最大距离，初始为 0；
- `ans[u]`：路径**必须包含 `u`**且以 `u` 为**最高点**时（最多往两条支路走）的最大路径长度，初始为 0。

孩子信息收集完后，**先更新 `ans` 再更新 `dist`**（保证"两条支路"拼起来时，`dist[u]` 还代表"之前某一条支路"的最优值）：

```

1 ans[u] = max(ans[u], dist[u] + dist[v] + weight[e]);
2 dist[u] = max(dist[u], dist[v] + weight[e]);

```

最终答案 = 所有 `ans[i]` 的最大值。

坑点

- `dist` 初始必须是 0（可以"一条边都不选"，自己单独待着，尤其有负边时）。
- 顺序不能反：先 `ans` 后 `dist`。

```

1 int dist[MAXN], ans[MAXN];
2
3 void dp(int u, int f) {
4     for (int e = head[u], v; e != 0; e = nxt[e]) {
5         v = to[e];
6         if (v != f) dp(v, u);
7     }
8     for (int e = head[u], v; e != 0; e = nxt[e]) {
9         v = to[e];
10        if (v != f) {
11            ans[u] = max(ans[u], dist[u] + dist[v] + weight[e]);
12            dist[u] = max(dist[u], dist[v] + weight[e]);
13        }
14    }
15 }
16 // 答案 = max(ans[1..n])

```

写法 3：树形 DP 化简

`ans` 数组可以省掉：DP 过程中需要决策 `ans[u]` 的地方，直接去更新一个全局 `diameter`。

```
1  int dist[MAXN], diameter;    // diameter 初始 INT_MIN
2
3  void dp(int u, int f) {
4      for (int e = head[u], v; e != 0; e = nxt[e]) {
5          v = to[e];
6          if (v != f) dp(v, u);
7      }
8      for (int e = head[u], v; e != 0; e = nxt[e]) {
9          v = to[e];
10         if (v != f) {
11             diameter = max(diameter, dist[u] + dist[v] + weight[e]);
12             dist[u] = max(dist[u], dist[v] + weight[e]);
13         }
14     }
15 }
```

题目二：所有直径的公共部分 (P3304)

题面

给定一棵边权都为正的树，打印两行：直径长度、所有直径的公共部分有几条边。

核心思路

- 两次 DFS 得到一条直径 `start` $\rightarrow$ `fin` 以及 `last` (可从 `fin` 跳回 `start`)、`dist` (从 `start` 出发的距离)。
- 对直径上每个点 `i`，求 `maxDistanceExceptDiameter(i)`：不经过直径路径上任何点能走出的最大距离（即该点挂着的“侧枝”最远能走多远）。
- 从 `fin` 往 `start` 跳，用结论判断公共部分的左右端点：
 - 若 `maxDist == dist[i]` (`i` 到 `start` 的距离)，满足条件；**最晚**满足的点是公共部分最右点 `r`。
 - 若 `maxDist == diameter - dist[i]` (`i` 到 `fin` 的距离)，满足条件；**最早**满足的点是公共部分最左点 `l`。
- `l == r` 说明所有直径只共用一个点，公共边数为 0；否则从 `l` 沿 `last` 跳到 `r` 数边数。

坑点

- `maxDistanceExceptDiameter` 里两个限制：不能走向 `diameterPath[v] == true` 的节点，也不能走回头路 (`v != f`)。
- 求 `maxDist` 时以 `long long` 存储 (`diameter - dist[i]` 可能较大)。
- 递归版会爆栈，需改迭代版 (见讲解118)。

```
1  const int MAXN = 200001;
2  int n, cnt, start, fin;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1], weight[MAXN << 1];
4  long long dist[MAXN], diameter;
```

```

5  int last[MAXN], commonEdges;
6  bool diameterPath[MAXN];
7
8  long long maxDistanceExceptDiameter(int u, int f, long long c) {
9      long long ans = c;
10     for (int e = head[u], v; e != 0; e = nxt[e]) {
11         v = to[e];
12         if (!diameterPath[v] && v != f)
13             ans = max(ans, maxDistanceExceptDiameter(v, u, c + weight[e]));
14     }
15     return ans;
16 }
17
18 void compute() {
19     road(); // 两次DFS得到 start/fin/diameter
20     for (int i = fin; i != 0; i = last[i]) diameterPath[i] = true;
21     int l = start, r = fin;
22     long long maxDist;
23     for (int i = last[fin]; i != start; i = last[i]) {
24         maxDist = maxDistanceExceptDiameter(i, 0, 0);
25         if (maxDist == diameter - dist[i]) r = i; // 最右
26         if (maxDist == dist[i] && l == start) l = i; // 最左
27     }
28     if (l == r) commonEdges = 0;
29     else {
30         commonEdges = 1;
31         for (int i = last[r]; i != l; i = last[i]) commonEdges++;
32     }
33 }

```

题目三：造公园 (P2195)

题面

n 个节点、 m 条边（边权都是 1），构成一个森林（多个连通区，每个连通区是树）。 q 次操作：

- 1 x ：返回 x 所在连通区的直径长度；
- 2 $x y$ ：若已连通则忽略；否则把两块连通区连通，使合并后的大区域直径最小（不要求输出方案）。

核心思路

- **并查集 + 集合打标签**：每个连通区用代表节点存直径 `diameter[代表]`。
- 建图时对每条边 u 、 v 做 `father[find(u)] = find(v)`；然后对每个“根代表”（ $i == \text{father}[i]$ ）跑一遍**树形 DP**求该连通区的直径（边权 1）。
- **合并公式**：两区域直径分别为 a 、 b ，新直径 = $\max((a+1)/2 + (b+1)/2 + 1, a, b)$ 。前一项是“卡着两个直径中点连一条边”形成的新直径，后两项是各自旧直径可能更大。

坑点

- 树形 DP 求直径只求长度即可，用 `dist[u] + dist[v] + 1` 决策。
- 合并时注意运算符顺序：`(diameter[x]+1)/2 + (diameter[y]+1)/2 + 1` 是整数除法向上取整。
- `diameter` 数组只在**代表节点**上有意义，非代表节点的值无用。

```

1  const int MAXN = 300001;
2  int cnt;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
4  int father[MAXN];      // 并查集
5  int dist[MAXN];        // 树形DP: 从u往下走最大距离
6  int diameter[MAXN];     // 代表节点才有效
7
8  int find(int i) { return i != father[i] ? father[i] = find(father[i]) : father[i]; }
9
10 void dp(int u, int f) {
11     for (int e = head[u], v; e != 0; e = nxt[e]) {
12         v = to[e];
13         if (v != f) dp(v, u);
14     }
15     for (int e = head[u], v; e != 0; e = nxt[e]) {
16         v = to[e];
17         if (v != f) {
18             diameter[u] = max(diameter[u], max(diameter[v], dist[u] + dist[v] + 1));
19             dist[u] = max(dist[u], dist[v] + 1);
20         }
21     }
22 }
23 // 合并: father[x]=y; diameter[y]=max((diameter[x]+1)/2+(diameter[y]+1)/2+1,
    max(diameter[x],diameter[y]));

```

题目四：巡逻 (P3629)

题面

n 个节点、 $n-1$ 条边（边权都是 1）的树，警车从 1 号点出发走遍所有边再回到 1 号点。可新建 k （1 或 2）条边，每条新建道路必须**恰好经过一次**，使经过的边数最少，返回最小值。

核心思路

- 不新建道路：每条边走两次，共 $2*(n-1)$ 。
- $k=1$ ：新建一条边连起直径两端，直径上的边都少走一次，答案 = $2*(n-1) - \text{diameter1} + 1$ 。
- $k=2$ ：先求直径 diameter1 ，把直径上所有点标记 diameterPath ；把**直径上的边权改成 -1**（其余仍为 1），再用**树形 DP** 求第二遍直径 diameter2 （负边只能用树形 DP）。答案 = $2*(n-1) - \text{diameter1} - \text{diameter2} + 2$ 。这样公共边的影响被自动抵消掉。

坑点

- 第二次求直径有负边，不能用两遍 DFS，必须树形 DP。
- 边权改成 -1 的方法：DP 里 $w = (\text{diameterPath}[u] \ \&\& \ \text{diameterPath}[v]) ? -1 : 1$ （两个端点都在直径上才算直径边）。
- $k=1$ 时无需跑第二遍。

```

1  int n, k, start, fin, diameter1, diameter2;
2  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
3  int dist[MAXN], last[MAXN];
4  bool diameterPath[MAXN];

```

```

5
6 void dp(int u, int f) { // 树形DP第二次求直径
7     for (int e = head[u], v; e != 0; e = nxt[e]) {
8         v = to[e];
9         if (v != f) dp(v, u);
10    }
11    for (int e = head[u], v, w; e != 0; e = nxt[e]) {
12        v = to[e];
13        if (v != f) {
14            w = diameterPath[u] && diameterPath[v] ? -1 : 1;
15            diameter2 = max(diameter2, dist[u] + dist[v] + w);
16            dist[u] = max(dist[u], dist[v] + w);
17        }
18    }
19 }
20
21 int compute() {
22     road(); // 两遍DFS求 diameter1、路径
23     if (k == 1) return 2 * (n - 1) - diameter1 + 1;
24     for (int i = fin; i != 0; i = last[i]) diameterPath[i] = true;
25     memset(dist + 1, 0, n * sizeof(int));
26     dp(1, 0);
27     return 2 * (n - 1) - diameter1 - diameter2 + 2;
28 }

```

题目五：消防（P2491）

题面

n 个节点、 $n-1$ 条边（边权非负）的树，每个节点是居民点。给定非负整数 s ，可在树上选一条长度不超过 s 的路径建消防站，每个居民去往路径上最近的消防站。求最远居民到消防站的最短距离。

核心思路

- 贪心结论：消防站路径一定建在直径上。
- 两次 DFS 求直径与路径信息；对直径上每个点 i 求 $\text{maxDist}[i]$ = 不经过直径路径上任何点能走出的最大距离（即 i 的侧枝最远距离）。
- 在直径上滑一个长度 $\leq s$ 的窗口（左闭右开 $[l, r)$ ），用单调队列维护窗口内 maxDist 的最大值。每个窗口的答案是三部分取最大：
 - 窗口左侧最远点到窗口左端点的距离 suml ；
 - 窗口内各点 maxDist 的最大值（单调队列队头）；
 - 窗口右侧最远点到窗口右端点的距离 $\text{diameter} - \text{sumr}$ 。
- 所有窗口答案取最小值返回。

坑点

- 必须会单调队列维护窗口内最大值（讲解054），否则窗口滑动时无法 $O(1)$ 得到 maxDist 最大值。
- 窗口是左闭右开 $[l, r)$ ： r 是"右边界再下一个点"， $\text{sumr} - \text{suml}$ 是当前窗口长度，再加 $\text{pred}[r]$ 判断能否继续扩。
- 实际代码是从 end 往 start 跳（因为只有 last 数组能往回跳），与图解方向相反，思路无区别。

- `pred[i]` 表示直径上 `i` 与前一个点之间的边长 (以 `start` 为根) , 用于计算窗口长度。
- 两遍 DFS 的递归会爆栈, 要改成迭代; `maxDistanceExceptDiameter` 递归也能过, 不用改。

```

1  const int MAXN = 300001;
2  int n, s, start, fin, diameter;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1], weight[MAXN << 1];
4  int dist[MAXN], last[MAXN], pred[MAXN], maxDist[MAXN];
5  bool diameterPath[MAXN];
6  int que[MAXN];    // 单调队列维护窗口内maxDist最大值
7
8  int maxDistanceExceptDiameter(int u, int f, int c) {
9      int ans = c;
10     for (int e = head[u], v; e != 0; e = nxt[e]) {
11         v = to[e];
12         if (!diameterPath[v] && v != f)
13             ans = max(ans, maxDistanceExceptDiameter(v, u, c + weight[e]));
14     }
15     return ans;
16 }
17
18 int compute() {
19     int suml = 0, sumr = 0, h = 0, t = 0, ans = INT_MAX;
20     // 窗口[l, r), 左闭右开; 从end往start跳
21     for (int l = fin, r = fin; l != 0; l = last[l]) {
22         while (r != 0 && sumr - suml + pred[r] <= s) {
23             while (h < t && maxDist[que[t - 1]] <= maxDist[r]) t--;
24             sumr += pred[r];
25             que[t++] = r;
26             r = last[r];
27         }
28         ans = min(ans, max(max(suml, maxDist[que[h]]), diameter - sumr));
29         if (que[h] == l) h++;
30         suml += pred[l];
31     }
32     return ans;
33 }

```